1.3 Expressions, Operators, and Precedence

In the previous section, we demonstrated how names can be used to identify existing objects, and how literals and constructors can be used to create instances of built-in classes. Existing values can be combined into larger syntactic ***expressions*** using a variety of special symbols and keywords known as ***operators***. The semantics of an operator depends upon the type of its operands. For example, when a and b are numbers, the syntax a + b indicates addition, while if a and b are strings, the operator indicates concatenation. In this section, we describe Python's operators in various contexts of the built-in types.

We continue, in Section 1.3.1, by discussing ***compound expressions***, such as a + b * c, which rely on the evaluation of two or more operations. The order in which the operations of a compound expression are evaluated can affect the overall value of the expression. For this reason, Python defines a specific order of precedence for evaluating operators, and it allows a programmer to override this order by using explicit parentheses to group subexpressions.

Logical Operators

Python supports the following keyword operators for Boolean values:

not	unary negation
and	conditional and
or	conditional or

The **and** and **or** operators ***short-circuit***, in that they do not evaluate the second operand if the result can be determined based on the value of the first operand. This feature is useful when constructing Boolean expressions in which we first test that a certain condition holds (such as a reference not being **None**), and then test a condition that could have otherwise generated an error condition had the prior test not succeeded.

Equality Operators

Python supports the following operators to test two notions of equality:

is	same identity
is not	different identity
==	equivalent
!=	not equivalent

The expression a **is** b evaluates to **True**, precisely when identifiers a and b are aliases for the *same* object. The expression a == b tests a more general notion of equivalence. If identifiers a and b refer to the same object, then a == b should also evaluate to **True**. Yet a == b also evaluates to **True** when the identifiers refer to

different objects that happen to have values that are deemed equivalent. The precise notion of equivalence depends on the data type. For example, two strings are considered equivalent if they match character for character. Two sets are equivalent if they have the same contents, irrespective of order. In most programming situations, the equivalence tests == and != are the appropriate operators; use of **is** and **is not** should be reserved for situations in which it is necessary to detect true aliasing.

Comparison Operators

Data types may define a natural order via the following operators:

$<$	less than
$<=$	less than or equal to
$>$	greater than
$>=$	greater than or equal to

These operators have expected behavior for numeric types, and are defined lexicographically, and case-sensitively, for strings. An exception is raised if operands have incomparable types, as with `5 < 'hello'`.

Arithmetic Operators

Python supports the following arithmetic operators:

$+$	addition
$-$	subtraction
$*$	multiplication
$/$	true division
$//$	integer division
%	the modulo operator

The use of addition, subtraction, and multiplication is straightforward, noting that if both operands have type int, then the result is an int as well; if one or both operands have type float, the result will be a float.

Python takes more care in its treatment of division. We first consider the case in which both operands have type int, for example, the quantity 27 divided by 4. In mathematical notation, $27 \div 4 = 6\frac{3}{4} = 6.75$. In Python, the / operator designates ***true division***, returning the floating-point result of the computation. Thus, `27 / 4` results in the float value 6.75. Python supports the pair of operators // and % to perform the integral calculations, with expression `27 // 4` evaluating to int value 6 (the mathematical *floor* of the quotient), and expression `27 % 4` evaluating to int value 3, the remainder of the integer division. We note that languages such as C, C++, and Java do not support the // operator; instead, the / operator returns the truncated quotient when both operands have integral type, and the result of true division when at least one operand has a floating-point type.

Python carefully extends the semantics of // and % to cases where one or both operands are negative. For the sake of notation, let us assume that variables n and m represent respectively the ***dividend*** and ***divisor*** of a quotient $\frac{n}{m}$, and that q = n // m and r = n % m. Python guarantees that q * m + r will equal n. We already saw an example of this identity with positive operands, as $6*4+3=27$. When the divisor m is positive, Python further guarantees that $0 \leq r < m$. As a consequence, we find that -27 // 4 evaluates to -7 and -27 % 4 evaluates to 1, as $(-7)*4+1=-27$. When the divisor is negative, Python guarantees that $m < r \leq 0$. As an example, 27 // -4 is -7 and 27 % -4 is -1, satisfying the identity $27=(-7)*(-4)+(-1)$.

The conventions for the // and % operators are even extended to floating-point operands, with the expression q = n // m being the integral floor of the quotient, and r = n % m being the "remainder" to ensure that q * m + r equals n. For example, 8.2 // 3.14 evaluates to 2.0 and 8.2 % 3.14 evaluates to 1.92, as $2.0*3.14+1.92=8.2$.

Bitwise Operators

Python provides the following bitwise operators for integers:

~	bitwise complement (prefix unary operator)
&	bitwise and
|	bitwise or
^	bitwise exclusive-or
<<	shift bits left, filling in with zeros
>>	shift bits right, filling in with sign bit

Sequence Operators

Each of Python's built-in sequence types (**str**, **tuple**, and **list**) support the following operator syntaxes:

s[j]	element at index *j*
s[start:stop]	slice including indices [start,stop)
s[start:stop:step]	slice including indices start, start + step, start + 2*step, ..., up to but not equalling or stop
s + t	concatenation of sequences
k * s	shorthand for s + s + s + ... (k times)
val **in** s	containment check
val **not in** s	non-containment check

Python relies on ***zero-indexing*** of sequences, thus a sequence of length n has elements indexed from 0 to $n-1$ inclusive. Python also supports the use of ***negative indices***, which denote a distance from the end of the sequence; index -1 denotes the last element, index -2 the second to last, and so on. Python uses a ***slicing***

notation to describe subsequences of a sequence. Slices are described as half-open intervals, with a start index that is included, and a stop index that is excluded. For example, the syntax `data[3:8]` denotes a subsequence including the five indices: $3, 4, 5, 6, 7$. An optional "step" value, possibly negative, can be indicated as a third parameter of the slice. If a start index or stop index is omitted in the slicing notation, it is presumed to designate the respective extreme of the original sequence.

Because lists are mutable, the syntax `s[j] = val` can be used to replace an element at a given index. Lists also support a syntax, **del** `s[j]`, that removes the designated element from the list. Slice notation can also be used to replace or delete a sublist.

The notation `val` **in** `s` can be used for any of the sequences to see if there is an element equivalent to `val` in the sequence. For strings, this syntax can be used to check for a single character or for a larger substring, as with `'amp'` **in** `'example'`.

All sequences define comparison operations based on ***lexicographic order***, performing an element by element comparison until the first difference is found. For example, [5, 6, 9] < [5, 7] because of the entries at index 1. Therefore, the following operations are supported by sequence types:

`s == t`	equivalent (element by element)
`s != t`	not equivalent
`s < t`	lexicographically less than
`s <= t`	lexicographically less than or equal to
`s > t`	lexicographically greater than
`s >= t`	lexicographically greater than or equal to

Operators for Sets and Dictionaries

Sets and frozensets support the following operators:

`key` **in** `s`	containment check
`key` **not in** `s`	non-containment check
`s1 == s2`	s1 is equivalent to s2
`s1 != s2`	s1 is not equivalent to s2
`s1 <= s2`	s1 is subset of s2
`s1 < s2`	s1 is proper subset of s2
`s1 >= s2`	s1 is superset of s2
`s1 > s2`	s1 is proper superset of s2
`s1 | s2`	the union of s1 and s2
`s1 & s2`	the intersection of s1 and s2
`s1 - s2`	the set of elements in s1 but not s2
`s1 ^ s2`	the set of elements in precisely one of s1 or s2

Note well that sets do not guarantee a particular order of their elements, so the comparison operators, such as <, are not lexicographic; rather, they are based on the mathematical notion of a subset. As a result, the comparison operators define

a partial order, but not a total order, as disjoint sets are neither "less than," "equal to," or "greater than" each other. Sets also support many fundamental behaviors through named methods (e.g., add, remove); we will explore their functionality more fully in Chapter 10.

Dictionaries, like sets, do not maintain a well-defined order on their elements. Furthermore, the concept of a subset is not typically meaningful for dictionaries, so the dict class does not support operators such as $<$. Dictionaries support the notion of equivalence, with d1 $==$ d2 if the two dictionaries contain the same set of key-value pairs. The most widely used behavior of dictionaries is accessing a value associated with a particular key k with the indexing syntax, d[k]. The supported operators are as follows:

d[key]	value associated with given key
d[key] = value	set (or reset) the value associated with given key
del d[key]	remove key and its associated value from dictionary
key **in** d	containment check
key **not in** d	non-containment check
d1 == d2	d1 is equivalent to d2
d1 != d2	d1 is not equivalent to d2

Dictionaries also support many useful behaviors through named methods, which we explore more fully in Chapter 10.

Extended Assignment Operators

Python supports an extended assignment operator for most binary operators, for example, allowing a syntax such as count += 5. By default, this is a shorthand for the more verbose count = count + 5. For an immutable type, such as a number or a string, one should not presume that this syntax changes the value of the existing object, but instead that it will reassign the identifier to a newly constructed value. (See discussion of Figure 1.3.) However, it is possible for a type to redefine such semantics to mutate the object, as the list class does for the += operator.

```
alpha = [1, 2, 3]
beta = alpha                # an alias for alpha
beta += [4, 5]              # extends the original list with two more elements
beta = beta + [6, 7]        # reassigns beta to a new list [1, 2, 3, 4, 5, 6, 7]
print(alpha)                # will be [1, 2, 3, 4, 5]
```

This example demonstrates the subtle difference between the list semantics for the syntax beta += foo versus beta = beta + foo.

1.3.1 Compound Expressions and Operator Precedence

Programming languages must have clear rules for the order in which compound expressions, such as $5 + 2 * 3$, are evaluated. The formal order of precedence for operators in Python is given in Table 1.3. Operators in a category with higher precedence will be evaluated before those with lower precedence, unless the expression is otherwise parenthesized. Therefore, we see that Python gives precedence to multiplication over addition, and therefore evaluates the expression $5 + 2 * 3$ as $5 + (2 * 3)$, with value 11, but the parenthesized expression $(5 + 2) * 3$ evaluates to value 21. Operators within a category are typically evaluated from left to right, thus $5 - 2 + 3$ has value 6. Exceptions to this rule include that unary operators and exponentiation are evaluated from right to left.

Python allows a ***chained assignment***, such as `x = y = 0`, to assign multiple identifiers to the rightmost value. Python also allows the ***chaining*** of comparison operators. For example, the expression `1 <= x + y <= 10` is evaluated as the compound `(1 <= x + y)` **and** `(x + y <= 10)`, but without computing the intermediate value `x + y` twice.

Operator Precedence		
	Type	**Symbols**
1	member access	expr.member
2	function/method calls container subscripts/slices	expr(...) expr[...]
3	exponentiation	**
4	unary operators	+expr, −expr, ~expr
5	multiplication, division	*, /, //, %
6	addition, subtraction	+, −
7	bitwise shifting	<<, >>
8	bitwise-and	&
9	bitwise-xor	^
10	bitwise-or	|
11	comparisons containment	**is**, **is not**, ==, !=, <, <=, >, >= **in**, **not in**
12	logical-not	**not** expr
13	logical-and	**and**
14	logical-or	**or**
15	conditional	val1 **if** cond **else** val2
16	assignments	=, +=, −=, *=, etc.

Table 1.3: Operator precedence in Python, with categories ordered from highest precedence to lowest precedence. When stated, we use expr to denote a literal, identifier, or result of a previously evaluated expression. All operators without explicit mention of expr are binary operators, with syntax expr1 operator expr2.